

ENGINEERING DECISION LOG

FREQUENCY / ProofForge Cyber • Team HM26-C3B5 • Phase 2

Q1 · WHAT WE BUILT — AND REJECTED

We built a proof pipeline in which candidate assessment answers and structured security findings enter the API; server-side scoring, evidence hashing and a human reviewer's decision turn them into section scores and verified findings; recruiter routes expose sanitized proof for discovery. Assessment answer keys stay on the server. A finding remains pending until a reviewer verifies it, rejects it or requests changes. The recruiter-facing capability and confidence signals are derived from evidence; claimed tools do not establish skill. The current interface is partly a demonstration: several candidate and recruiter views use synthetic browser state, while the API persists assessment section scores, findings, reviewer decisions, defense answers, rubric values and invitations.

We seriously considered an automated LLM grader. It looked attractive because it could return feedback immediately and reduce reviewer queue time. We rejected it as the authority for hiring decisions: exploit evidence is adversarial input, and a generated score would be difficult to reproduce and defend. The implemented reviewer suggestion is deterministic demo content, not a runtime model.

Q2 · WHY — AND THE COST WE ACCEPT

Across **trust**, **repeatability**, **build scope** and **latency**, human verification with deterministic API rules was the better fit. Reviewers can inspect reproduction steps and explain a decision; server-side answer keys and explicit ownership checks constrain basic tampering. The approach was feasible to implement without collecting labeled examples, hosting a model or designing prompt-injection defenses. A language model might summarize reports faster, but it can miss context or follow instructions embedded in hostile evidence. We therefore keep final verification and hiring decisions with people.

We knowingly accept slower verification: candidates wait for a reviewer, and reviewer capacity becomes a bottleneck. We also accept an incomplete product boundary: role pages are not yet fully wired to authenticated API records, and synthetic UI state must not be mistaken for persisted candidate evidence. This costs immediacy and end-to-end consistency. The MVP favors explainable records and human accountability over instant but less defensible automated judgments.

Q3 · WHAT BREAKS ACROSS MYSURU

Assume 50,000 candidates complete four assessment sections during a 20-minute placement window. The API stores one result per candidate per section, so that is **200,000 score submissions**, averaging about **167 requests per second** if evenly spread; a deadline rush would be higher. This is not two million question-result rows: each request scores ten answers and upserts one section result. Today scoring runs synchronously, then the audit helper commits separately. The Render blueprint also configures one Gunicorn worker and in-memory rate limiting. A burst can therefore exhaust request/DB capacity, while adding workers would make per-process rate limits inconsistent.

Our first change would be a durable submission/outbox queue with idempotency on (`candidate_id`, `section`), so requests can be accepted quickly and scoring/audit work drained at a controlled rate. We would move rate limits to shared Redis before horizontal scaling, then load-test peak bursts and tune worker and database capacity from measurements. This adds operational components and makes results briefly asynchronous; we accept that cost to protect the service during festival or campus-wide traffic.